COMMONWEALTH OF AUSTRALIA
Copyright Regulations 1969
This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice

Prepared by: [Arun Konagurthu]

FIT3155: Advanced Algorithms and Data Structures

Week 8: **Amortized analysis**

Faculty of Information Technology, Monash University

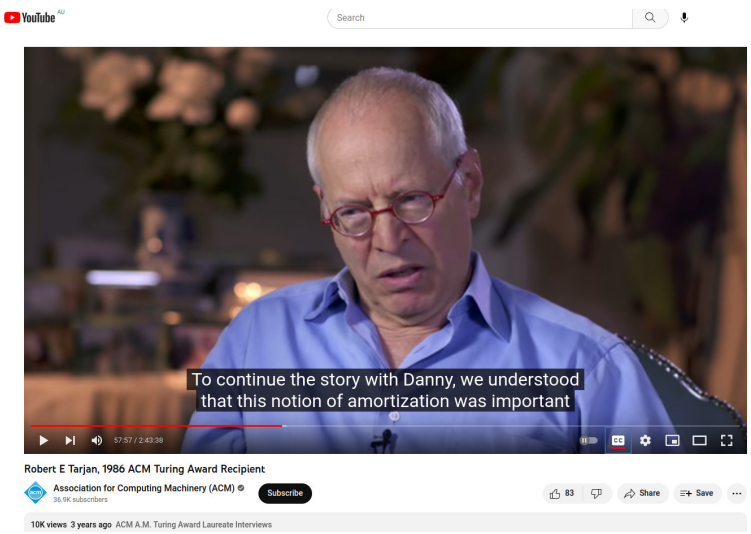
What is covered in this lecture?

Amortized analysis techniques

- Aggregate analysis
- Accounting method
- Potential method

Source material and recommended reading

- Cormen et al. "***Introduction to Algorithms***" (Chapter 17)
- Robert E. Tarjan "***Amortized Computational Complexity***" SIAM Journal on Algebraic and Discrete Methods, 6(2), 1985



Amortize?

https://www.merriam-webster.com/dictionary/amortize

Merriam-Webster Est. 1828 Dictionary | Dictionary | Thesaurus | amortize | Games & Quizzes | Thesaurus | Features

Definition

Did you know?

Example Sentences

Word History

Podcast

Entries Near

Show More

Save Word

amortize verb

am·or·tize ȧ-mər-ˌtīz also ə-mȯr-

amortized; amortizing

transitive verb

1 : to pay off (an obligation, such as a mortgage) gradually usually by periodic payments of principal and interest or by payments to a sinking fund

amortize a loan

2 : to gradually reduce or write off the cost or value of (something, such as an asset)

amortize goodwill

amortize machinery

amortizable ȧ-mər-ˌtī-zə-bəl also ə-mȯr- adjective

Quoting Bob Tarjan, this term is adapted for **computational complexity** to mean "*to average over time or, more precisely, to average the running times of operations in a sequence over that sequence.*"

Motivation for Amortization in computational complexity

- In a routine use of any **data structure**, a **sequence** of operations are performed on it.
- It is useful to find bounds on the **true** time/cost to perform a **sequence** of operations.

Motivation for Amortization in computational complexity

- Summing up the **worst-case** per-operation times over a sequence of operations gives an **unduly pessimistic** estimate.
- Summing up the **best-case** per-op times over a sequence of ops yields an **unreasonably optimistic** estimate.
- Summing up **average-case** times of operations remains **infeasible** and/or **inaccurate**.
  - characterizing probability distributions over sequences of operations is non-trivial and, even if one came up with one, can lead to unrealistic estimates.

What is Amortized analysis?

An **amortized analysis** is instead used to provide a more **realistic** way to reason about the running times per operation by analyzing the bounds on the time/cost of performing any sequence of operations. (We will see how in the upcoming slides)

- Amortized analysis is about the performance of an **operation**, but averaged over any sequence of operations
- A more realistic **per-operation** time/cost can be derived, and sometimes shown to be smaller than the operation's **worst-case** behaviour.
- This is achieved by **bounding** the **true** time/cost of any possible **sequence** of operations.

Amortized analysis: three commonly used techniques

1. **Aggregate method** – a **mathematician's** view of amortization.
2. **Accounting method** – a **banker's** view of amortization.
3. **Potential method** – a **physicist's** view of amortization.

We will use a simple data structure to illustrate each of these methods.

Understanding the three methods of amortization using a simple data structure: k-bit binary counter

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

A k-bit binary counter data structure

- The data structure is an boolean array of size k to store any number x in a **binary** (base-2) form.
- Let $A[0 \ldots k-1]$ be the array data structure, storing bits from the lowest ($A[0]$) to the highest ($A[k-1]$) bit order.
- The data structure is initialized to $x = 0$ in its initial state.
- This data structure supports a single operation, **increment**. This adds 1 (modulo 2^k) to the current value in the counter – see pseudocode below

The increment operation

```
b = 0
while b < k and A[b] == 1:
    A[b] = 0 /*unset*/
    b = b+1
if b < k:
    A[b] = 1 /*set*/
```

The logic of the **increment** operation

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

```
1 b = 0
2 while b < k and A[b] == 1:
3     A[b] = 0 /*unset*/
4     b = b+1
5 if b < k:
6     A[b] = 1 /*set*/
```

- In each iteration of the while loop, the logic involves exploring if the bit of order b is set, i.e. $A[b] \stackrel{?}{=} 1$.
- If that bit is set (i.e. $A[b] = 1$)
  - flip/unset that bit to 0 in the array (line 3), and
  - carry forward 1 (to the next order bit b+1, handled in the next iteration of the while loop).

Worst-case analysis of increment operation

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

- The **true cost** of each **increment** operation is **linear** in the **runlength** of set (1) bits before the first unset (0) bit is observed. (e.g. the counter values $\{5, 6, 7, 13, 15\}$ have **runlengths** $\{1, 0, 3, 1, 4\}$, resp.)
  - This **runlength** dictates the number of times the while loop executes, and hence its runtime (i.e. true operation cost).
- Clearly, the worst-case **runlength** is k, so the worst-case time for that operation is $O(k)$.
- Thus, if one used the worst-case to analyse the cost of a sequence of n increments one would arrive at the pessimistic bound of $O(n \times k)$ time.

Now let's analyze the amortized cost using the three methods of amortization, after introducing each.

Method 1: Aggregate method of amortization

Aggregate method: Introduction

- This method can be used when the **true time/cost** over a **sequence** of operations can be more directly **bounded** in a mathematical way, which can then be **averaged out** over that sequence length.
- In other words, in an **aggregate analysis**, one usually attempts to characterize directly a **bound** $T(n)$ on the total cost/time of a sequence of n operations of a data structure.
- If one reasons the bound for any possible sequence, the average cost per operation is estimated as $\frac{T(n)}{n}$.

Aggregate method of amortization on binary counter

Assume a sequence of n increments.

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

- There are $\sim \frac{n}{2}$ counter values with $A[0] = 0$ (**runlength**=0), requiring just setting A[0]. (1 unit of effort.)
- There are $\sim \frac{n}{4}$ counter values with $A[0] = 1$ but $A[1] = 0$ (**runlength**=1), requiring flipping $A[0]$ to 0 and setting $A[1]$ to 1. (2 units of effort.)
- There are $\sim \frac{n}{8}$ counter values with $A[0 \ldots 1] = 1$ but $A[2] = 0$ (**runlength**=2), requiring flipping $A[0 \ldots 1]$ to 0 and setting $A[2]$ to 1. (3 units of effort.)
- There are $\sim \frac{n}{16}$ counter values with $A[0 \ldots 2] = 1$ but $A[3] = 0$ (**runlength**=3), requiring flipping $A[0 \ldots 2]$ to 0 and setting $A[3]$ to 1 (4 units of effort.), and so on...

Summing total time over n increments

- $T = \frac{n}{2} + \frac{2n}{4} + \frac{3n}{8} + \frac{4n}{16} \ldots \leq 2n$ (watch lecture for proof)

Thus, total time over n increments is bounded by $O(n)$, implying $O(1)$-time amortized cost for each **increment**!

Aggregate method of amortization on **binary counter**

Assume a sequence of n increments.

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

- There are $\sim \frac{n}{2}$ counter values with $A[0] = 0$ (**runlength**=0), requiring just setting A[0]. (1 unit of effort.)
- There are $\sim \frac{n}{4}$ counter values with $A[0] = 1$ but $A[1] = 0$ (**runlength**=1), requiring flipping $A[0]$ to 0 and setting $A[1]$ to 1. (2 units of effort.)
- There are $\sim \frac{n}{8}$ counter values with $A[0 \ldots 1] = 1$ but $A[2] = 0$ (**runlength**=2), requiring flipping $A[0 \ldots 1]$ to 0 and setting $A[2]$ to 1. (3 units of effort.)
- There are $\sim \frac{n}{16}$ counter values with $A[0 \ldots 2] = 1$ but $A[3] = 0$ (**runlength**=3), requiring flipping $A[0 \ldots 2]$ to 0 and setting $A[3]$ to 1 (4 units of effort.), and so on...

Summing total time over n increments

- $T = \frac{n}{2} + \frac{2n}{4} + \frac{3n}{8} + \frac{4n}{16} \ldots \leq 2n$ (watch lecture for proof)

Thus, total time over n increments is bounded by $O(n)$, implying $O(1)$-time amortized cost for each **increment**!

Method 2: Accounting method of amortization

Accounting method (Amortized Analysis)

- Assume an imaginary 'coin-operated' data structure).
- Think of coins as buying **`credits`** to pay for some amount of work operating this data structure
- Each coin buys a **fixed amount of computation**.
- The **`true cost`** of each operation may vary.
- We can reason and assign some carefully chosen number of coins as part of each operation during this analysis – this is its **`amortized cost`**.
- Any extra coins (credit) can be **saved** and used to pay for future work.

Goal:

Show that any sequence of operations can be performed without running out of credits.

Accounting method: Key idea

- If we never go into debt, then:
- Total **actual cost** $\leq$ total **amortized cost**
- This holds for **any sequence of operations**.
- Credits are only an **analysis tool**:
  - They are **not implemented** in the data structure.

Allocation of 'coins' for key tasks of the **increment** operation in the **binary counter** data structure

The **increment** operation

```
1 b = 0
2 while b < k and A[b] == 1:
3     A[b] = 0 /*unset*/
4     b = b+1
5 if b < k:
6     A[b] = 1 /*set*/
```

- The increment operation consists of two atomic tasks:
  1. Setting a bit to 1 (line 6).
     - ★ This takes $O(1)$ time.
     - ★ Line 6 is executed at most once per increment.
  2. Unsetting bits to 0 (line 3).
     - ★ Only previously set bits are unset (line 3, via the while-condition on line 2).
     - ★ Line 3 may execute multiple times per increment until the while-condition becomes false.
     - ★ Each iteration of the loop (lines 2-4) costs $O(1)$ time.

Allocation of 'coins' for key tasks of the **increment** operation in the **binary counter** data structure

The **increment** operation

```
1 b = 0
2 while b < k and A[b] == 1:
3     A[b] = 0 /*unset*/
4     b = b+1
5 if b < k:
6     A[b] = 1 /*set*/
```

- Assume 1 coin pays for $O(1)$ work in any part of the **increment** operation.
- We assign $\lambda > 0$ coins whenever a bit is set (line 6).
- Setting a bit costs 1 coin, leaving $\lambda - 1$ coins as credit stored on that bit for future work.
- Can this stored credit pay for unsetting the bit later (during future increments)?
  - Yes. Each time a bit is reset, it is due to a while-loop iteration (lines 3–4), which costs $O(1)$ and is paid for by 1 coin.
  - Since each set bit carries $\lambda - 1$ credits, choosing $\lambda \geq 2$ ensures we never run out of credit.
- Hence, a sensible choice is $\lambda = 2$.

Accounting method applied on the binary counter

```
1 b = 0
2 while b < k and A[b] == 1:
3     A[b] = 0  /* unset */
4     b = b + 1
5 if b < k:
6     A[b] = 1  /* set */
```

- Assign amortized cost $\lambda = 2$ coins to setting a bit (line 6).
  - 1 coin pays for the actual write $0 \to 1$.
  - 1 coin is stored as credit on that bit.
- Assign 1 coin per loop iteration (lines 2-4).
  - This pays for unsetting a bit $1 \to 0$ (line 3).

- Each increment sets at most one bit (line 6).
- Each unset operation is paid for by credit stored on previously set bits.
- Hence no operation goes into debt (credit is always nonnegative).
- Therefore, each increment costs at most 2 **amortized** coins $\Rightarrow O(1)$ **amortized time**.
- Over n increments, total **amortized cost** is $O(n)$.

Method 3: Potential method of amortization

Historical note:

- The physicist's view of amortization was proposed by Daniel Sleator who was a PhD supervisor of Bob Tarjan.
- Together with Tarjan, they pioneered amortized analyses of data structures and algorithms.
- Important The banker's accounting and physicist's potential methods of amortization are essentially **equivalent**. The potential view provides a way to deal with fractional costs (for time), whereas the banker's view historically dealt with integer credits/debits. But important to note that both yield the same bounds on the true cost.

Potential method: Introduction

- Let D_0 be the initial state of some hypothetical data structure.
- Over n operations let the state of this data structure change as: $D_0 \to D_1 \to D_2 \to \ldots D_n$.
- Let τ_i be the actual time it takes to execute the i-th operation (changing the data structure from $D_{i-1} \to D_i$.

Potential method: Introduction

- Define a **potential function** Φ that maps each state D_i of the data structure to a real number $\Phi(D_i)$, called its **potential**.
  - A unit of potential represents **stored work (or credit)** that can pay for a **constant amount of computation**, sufficient to cover constant-time steps in operations.
  - **Intuition:** $\Phi(D_i)$ measures how much **future work has already been prepaid and stored as credit** in the current state.
- The **amortized cost/time** of the i-th operation is defined as:

$$a_i = \tau_i + \Phi(D_i) - \Phi(D_{i-1}).$$

  - **Intuition:** amortized cost = actual cost + change in stored credit.
    - ★ If potential increases, we are **prepaying credit for future operations**.
    - ★ If potential decreases, we are **using previously stored credit**.
- Summing over a sequence of n operations gives:

$$\sum_{i=1}^{n} a_i = \sum_{i=1}^{n} \tau_i + \Phi(D_n) - \Phi(D_0)$$

(To be derived in the lecture)

  - This shows that amortized analysis accounts for both **actual work** and changes in **stored credit**.

Potential method: Introduction (...cont'd)

- From the previous slide:

$$\sum_{i=1}^{n} a_i = \sum_{i=1}^{n} \tau_i + \Phi(D_n) - \Phi(D_0)$$

- In words, the total **amortized** cost equals total **actual** cost plus the net change in potential between final and initial states.
- The potential function $\Phi(\cdot)$ can be chosen freely depending on the problem (see next slide).

Choosing a potential function Φ

- An astute choice of Φ can yields an informative (tighter) amortized bound.
- In most cases (for convenience), the potential function is chosen such that where $\Phi(D_0) = 0$ and $\Phi(D_i) \geq 0$ for all i.
- While that is the matter of convenience, the key requirement is that $\Phi(D_n) - \Phi(D_0) \geq 0$ so that amortized cost upper bounds actual cost.
- Different valid choices of Φ can give different amortized bounds; the goal is to choose one that yields the tightest bound.

A **Potential function** for the **binary counter** data structure

- In the binary counter example, the data structure is an array of bits $A[0 \ldots k-1]$ of size k.
- Let $A_i[0 \ldots k-1]$ denote the state of the binary counter after the ith increment operation ($\forall i \geq 0$)).
- One could define a viable potential function as follows:

$$\Phi(A_i) = \texttt{bitsum}(A_i) = \sum_{b=0}^{k-1} A_i[b]$$

- With the above defined potential function:
  - $\Phi(A_0) = 0$.
  - $\Phi(A_i) \geq 0$ for all $i \geq 0$.
  - The maximum potential is k (when in some state, all k bits are set).

Potential method applied on the binary counter

We are using the potential function for this data structure specifically as

$$\Phi(A_i) = \texttt{bitsum}(A_i) = \sum_{b=0}^{k-1} A_i[b]$$

We saw on slide 23...

- ... the **amortized time** of an operation **in general** as:

$$a_i = \tau_i + \Phi(D_i) - \Phi(D_{i-1}).$$

- Over a sequence of n operations, we have the total amortized time as:

$$\sum_{i=1}^{n} a_i = \sum_{i=1}^{n} \tau_i + \Phi(D_n) - \Phi(D_0)$$

- Suppose that the i-th increment results in **unsetting** $(1 \to 0)$ some l trailing bits before potentially setting $(0 \to 1)$ the next most significant bit.
- The **true cost** τ_i is bounded by $l + 1$.

- Thus, **specifically for the binary counter**,

$$\Phi(A_i) = \begin{cases} \Phi(A_{i-1}) - l + 1, & \Phi(A_{i-1}) < k \\ \Phi(A_{i-1}) - l, & \Phi(A_{i-1}) = k \end{cases}$$

(Refer conditional statement on Line 5 of the pseudocode on slide 10).

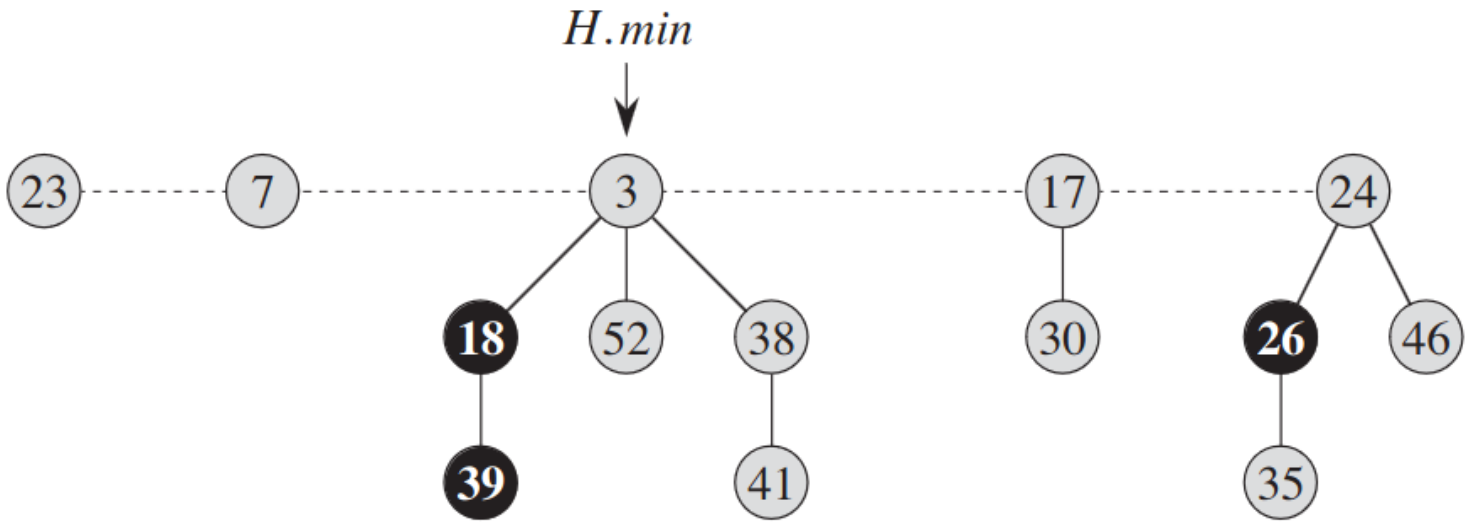
- The amortized cost of this operation under this potential is

$$\begin{aligned} a_i &= \tau_i + \Phi(A_i) - \Phi(A_{i-1}) \\ &\leq l + 1 + \Phi(A_{i-1}) - l + 1 - \Phi(A_{i-1}) \\ &\leq 2 = O(1). \end{aligned}$$

- Hence, the amortized bound on the true cost of a sequence of n **increment** operations is $O(n)$.

The performance of various **Fibonacci heap** operations using the **potential function** method

Defining a potential function for Fibonacci heap operations



- For a given Fibonacci heap H denote:
  - $\texttt{nTrees}(H)$ to mean the number of trees in the root list of H and
  - $\texttt{nMarked}(H)$ to mean the number of marked nodes in H.
- A **potential** $\Phi(H)$ of any Fibonacci heap H is chosen to be

$$\Phi(H) = \texttt{nTrees}(H) + 2(\texttt{nMarked}(H)).^*$$

- For example, the potential of the Fibonacci heap shown above is $\Phi(H) = 5 + 2(3) = 5 + 6 = 11$.

*(We will gain some intuition for this potential function in the upcoming slides.)

Amortized cost of Fibonacci heap's **insert** operation

Recall from slide 23...

- ... for a data strucuture D, **amortized time** is defined as:

$$a_i = \tau_i + \Phi(D_i) - \Phi(D_{i-1}).$$

- True time τ_i to perform any insert in the Fibonacci heap H is clearly $O(1)$.
- Inserting an element into H increases the number of trees (in the root level) by 1: $\texttt{nTrees}(H_i) = \texttt{nTrees}(H_{i-1}) + 1$.
- On the other hand, inserting an element into H does not change the number of marked nodes: $\texttt{nMarked}(H_i) = \texttt{nMarked}(H_{i-1})$.

- Hence, the potential difference going from $H_{i-1} \to H_i$ is:

$$\Delta\Phi = \Phi(H_i) - \Phi(H_{i-1}) = 1$$

- Thus the **amortized cost** of this operation under the defined potential (see slide 29) is:

$$\begin{aligned} a_i &= \tau_i + \Phi(H_i) - \Phi(H_{i-1}) \\ &= 1 + 1 \\ &= 2 \end{aligned}$$

- This gives an amortised cost of $O(1)$ for each insert.
- From the above, since $\Phi(H_0) = 0$, it can be shown that the sum of amortized costs is $O(n)$ and bounds the true cost of any sequence of n inserts.

Amortized cost of Fibonacci heap's **extract-min**

- Recall that **extract-min** extracts the root node pointed by H.min, promotes its children (trees) into the root-list and then consolidates the heap such there is at most one tree of any given degree.
- Assume **extract-min** changes the state of the heap from H_{i-1} into H_i.
- The true cost of **extract-min** is bounded by $\text{degree}_{\text{max}}(H_{i-1}) + \texttt{nTrees}(H_{i-1})$.
- The **potential** before extracting node pointed by H_{i-1}.min is

$$\Phi(H_{i-1}) = \texttt{nTrees}(H_{i-1}) + 2(\texttt{nMarked}(H_{i-1})).$$

- The **potential** after extracting is

$$\begin{aligned}\Phi(H_i) &= \texttt{nTrees}(H_i) + 2(\texttt{nMarked}(H_i)) \\ &\leq \text{degree}_{\text{max}}(H_i) + 1 + 2(\texttt{nMarked}(H_{i-1})).\end{aligned}$$

The above inequality holds because (i) the nTrees(H_i) in the worst case after consolidation is $\text{degree}_{\text{max}}(H_i) + 1$, and (ii) no nodes become newly marked during extract-min and potentially the min node that has been extracted could have been previously marked, making nMarked$(H_i) \leq$ nMarked(H_{i-1}).

- Thus, the **amortized cost** of extract-min is:

$$\begin{aligned}a_i &= \tau_i + \Phi(H_i) - \Phi(H_{i-1}) \\ &\leq \underbrace{\text{degree}_{\text{max}}(H_{i-1}) + \texttt{nTrees}(H_{i-1})}_{\tau_i} + \underbrace{\text{degree}_{\text{max}}(H_i) + 1 - \texttt{nTrees}(H_{i-1})}_{\Phi(H_i)-\Phi(H_{i-1})} \\ &\leq \text{degree}_{\text{max}}(H_i) + \text{degree}_{\text{max}}(H_{i-1}) + 1\end{aligned}$$

- Since $\text{degree}_{\text{max}}(H)$ grows $O(\log H.\text{size})$, the amortized cost of **extract-min** is $O(\log(H.\text{size}))$ (H.size is the number of elements in the heap.).

Amortized cost of Fibonacci heap's **decrease-key**

- Let **decrease-key** operation on the state H_{i-1} of the heap results in $m \geq 0$ nodes being promoted to the root level.
- The true cost $\tau_i = m$ (as the effort is dominated by m nodes being promoted to the root).
- The **potential** before decrease-key was called is:

$$\Phi(H_{i-1}) = \texttt{nTrees}(H_{i-1}) + 2(\texttt{nMarked}(H_{i-1})).$$

- The **potential** after decreasing the key is

$$\begin{aligned} \Phi(H_i) &= \texttt{nTrees}(H_i) + 2(\texttt{nMarked}(H_i)) \\ &\leq \texttt{nTrees}(H_{i-1}) + m + 2(\texttt{nMarked}(H_{i-1}) - (m-1) + 1). \end{aligned}$$

  (inequality holds because at least $m-1$ and at most m previously marked nodes that were promoted to the root will now be unmarked, and the final cascading cut could have marked its previously unmarked parent)
- Thus, the **amortized cost** of decrease-key is:

$$\begin{aligned} a_i &= \tau_i + \Phi(H_i) - \Phi(H_{i-1}) \\ &\leq \underbrace{m}_{\tau_i} + \underbrace{\texttt{nTrees}(H_{i-1}) + m + 2(\texttt{nMarked}(H_{i-1})) - 2m + 4}_{\Phi(H_i)} - \underbrace{\texttt{nTrees}(H_{i-1}) - 2(\texttt{nMarked}(H_{i-1}))}_{\Phi(H_{i-1})} \\ &\leq 4 \end{aligned}$$

- Thus, this gives the amortized cost of $O(1)$ per **decrease-key** operation.

Intuition behind the chosen potential for Fibonacci – continued

- All Fibonacci heap operations potentially affect the number of trees in the root level and in the case of decrease-key, the number of marked nodes over all in the heap.
- So a viable potential function involves those two criteria.
- Yet, it does not explain how those constant multipliers in the specified potential function were chosen.
- Assume the potential of the Fibonacci heap H were more generally defined as:

$$\Phi(H) = \lambda_1(\texttt{nTrees}(H)) + \lambda_2(\texttt{nMarked}(H))$$

Intuition behind the chosen potential for Fibonacci Heap – continued

Revisiting the amortized analysis of decrease-key with this general form:

- The **potential** before decrease-key was called is:

$$\Phi(H_{i-1}) = \lambda_1(\texttt{nTrees}(H_{i-1})) + \lambda_2(\texttt{nMarked}(H_{i-1})).$$

- The **potential** after decreasing the key is

$$\begin{aligned}\Phi(H_i) &= \lambda_1(\texttt{nTrees}(H_i)) + \lambda_2(\texttt{nMarked}(H_i)) \\ &\leq \lambda_1(\texttt{nTrees}(H_{i-1}) + m) + \lambda_2(\texttt{nMarked}(H_{i-1}) - (m-1) + 1).\end{aligned}$$

Thus, we have the amortized cost bounding the true cost as:

$$\begin{aligned}a_i &= \tau_i + \Phi(H_i) - \Phi(H_{i-1}) \\ &\leq m + \lambda_1(\texttt{nTrees}(H_{i-1}) + m) + \lambda_2(\texttt{nMarked}(H_{i-1}) - (m-1) + 1) \\ &\quad -\lambda_1(\texttt{nTrees}(H_{i-1})) - \lambda_2(\texttt{nMarked}(H_{i-1})) \\ &\leq (1 + \lambda_1 - \lambda_2)m + 2\lambda_2\end{aligned}$$

This is not an informative amortized cost, as the r.h.s depends on m (the number of marked nodes that were promoted to the root level, which varies per operation). However, if one were to choose values for λ_1 and λ_2 more astutely such that $1 + \lambda_1 - \lambda_2 = 0$, one could make that term involving m disappear, giving a $O(1)$ cost. Thus, $\lambda_1 = 1$ and $\lambda_2 = 2$ becomes a natural choice.

Next week: B-trees